



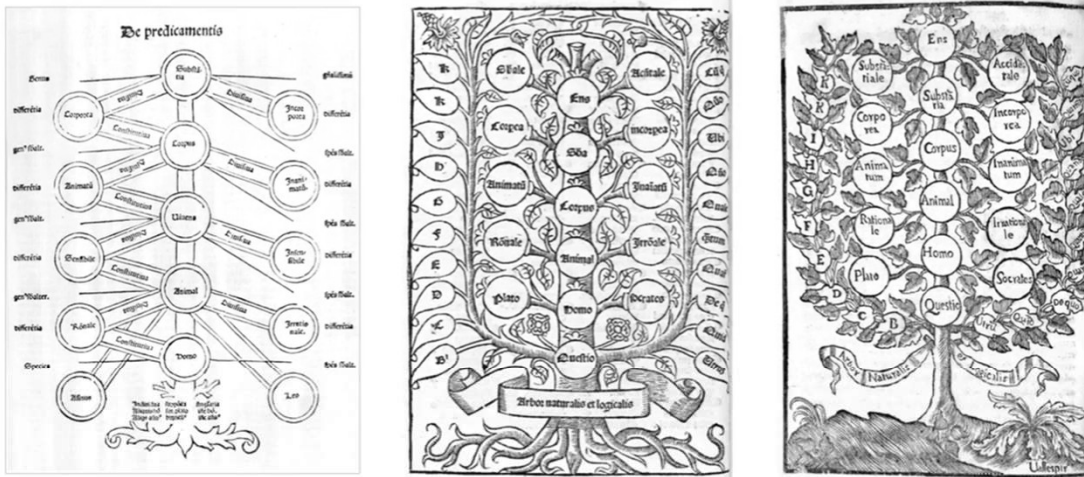
Generative AI

Knowledge Graphs, Graph RAG

Knowledge Graphs, Graph RAG

- Knowledge Graphs
- Graph Database (Neo4j)
- Cypher query language
- Graph RAG / KG RAG

Knowledge Graphs have been a key part of mapping data, throughout human history



Porphyrian Tree (oldest Tree of Knowledge)

A visual history of human knowledge

2,077,878 plays ⓘ | Manuel Lima | TED2015 • March 2015

<https://www.youtube.com/watch?v=BQZKs75RMqM>

https://www.ted.com/talks/manuel_lima_a_visual_history_of_human_knowledge?language=en

Graph Theory origins

1736 – Leonhard Euler

Which route would allow someone
to cross all 7 bridges



without
crossing

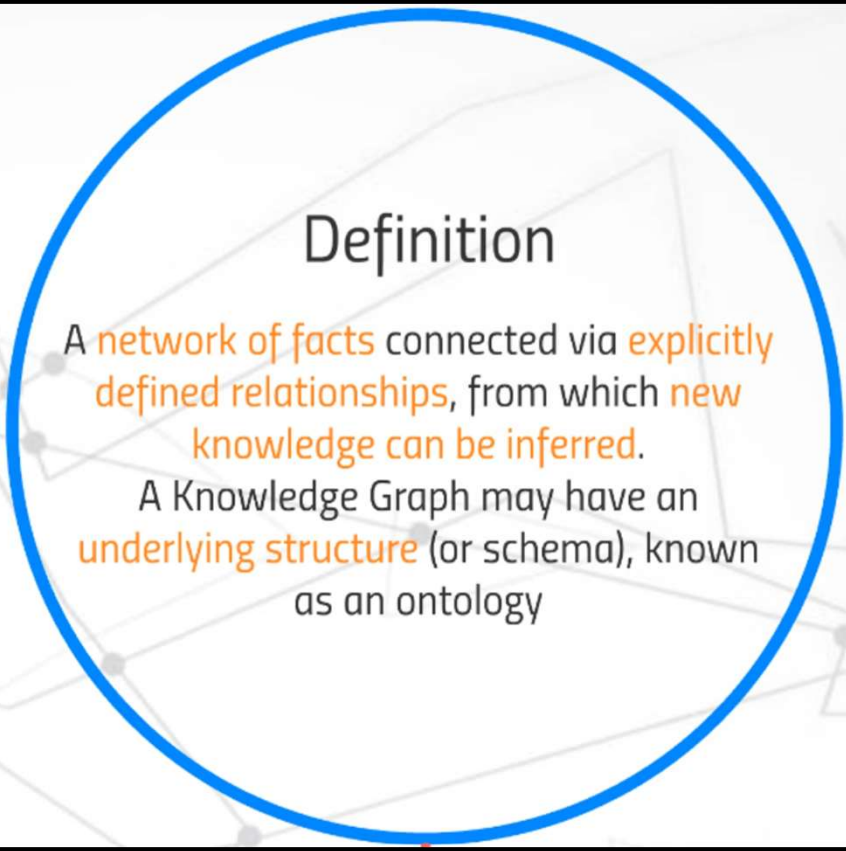
any of them more than once?

[Seven Bridges of Königsberg – Wikipedia](#)

[How the Königsberg bridge problem changed mathematics - Dan Van der Vieren](#)

<https://www.youtube.com/watch?v=ycRuO-u6rt8>

Knowledge Graph - definition

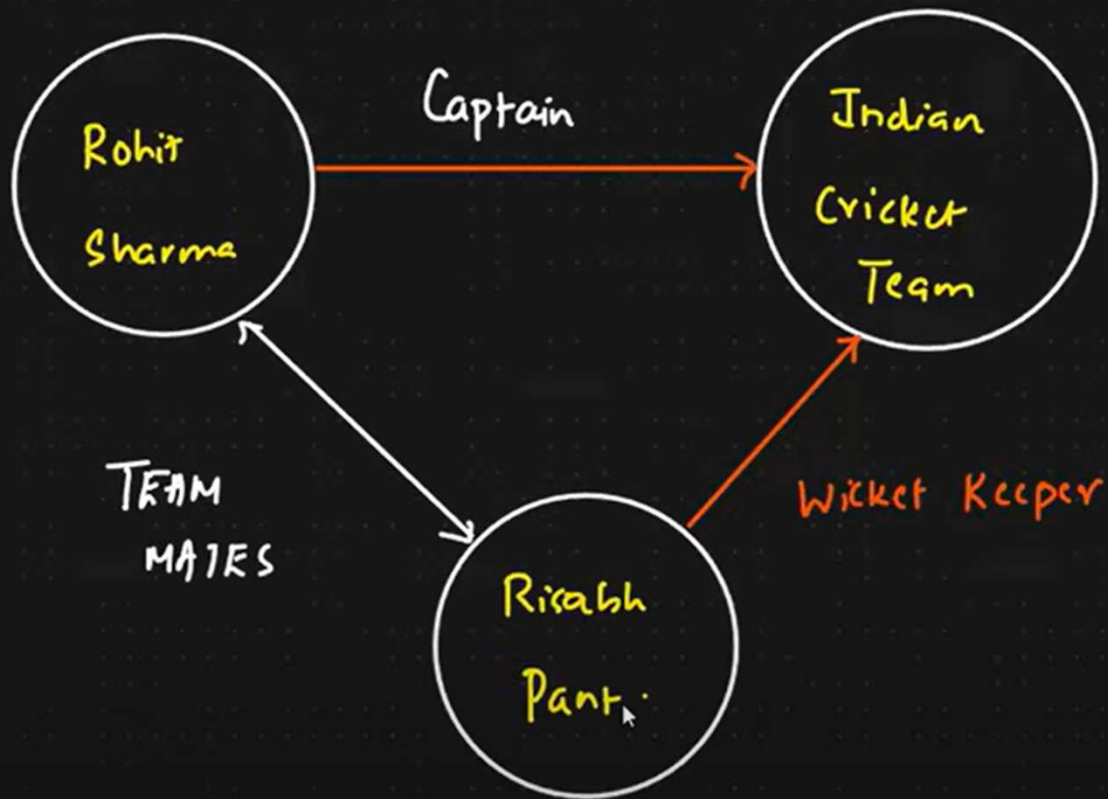


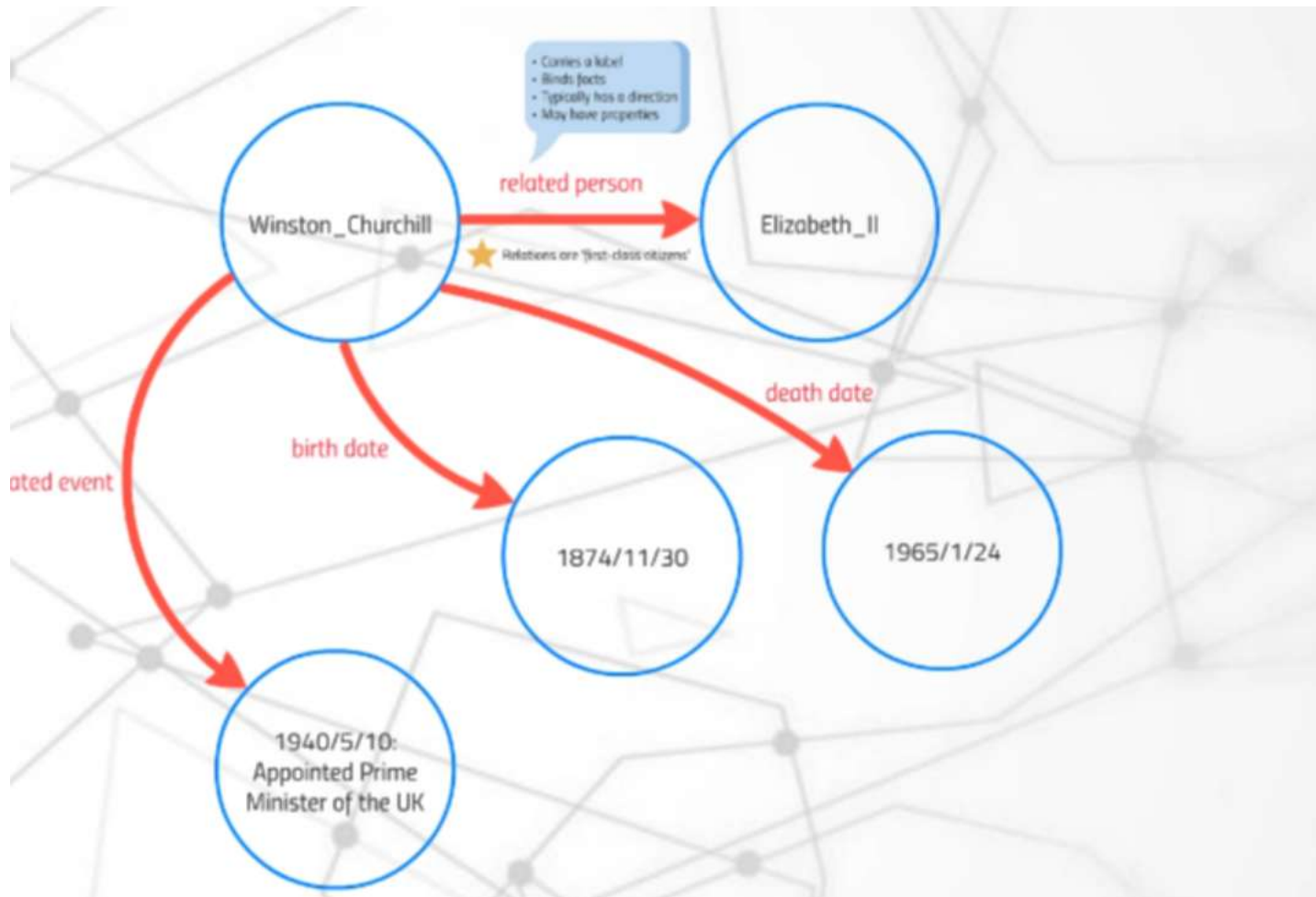
Definition

A **network of facts** connected via **explicitly defined relationships**, from which **new knowledge can be inferred**.

A Knowledge Graph may have an **underlying structure** (or schema), known as an ontology

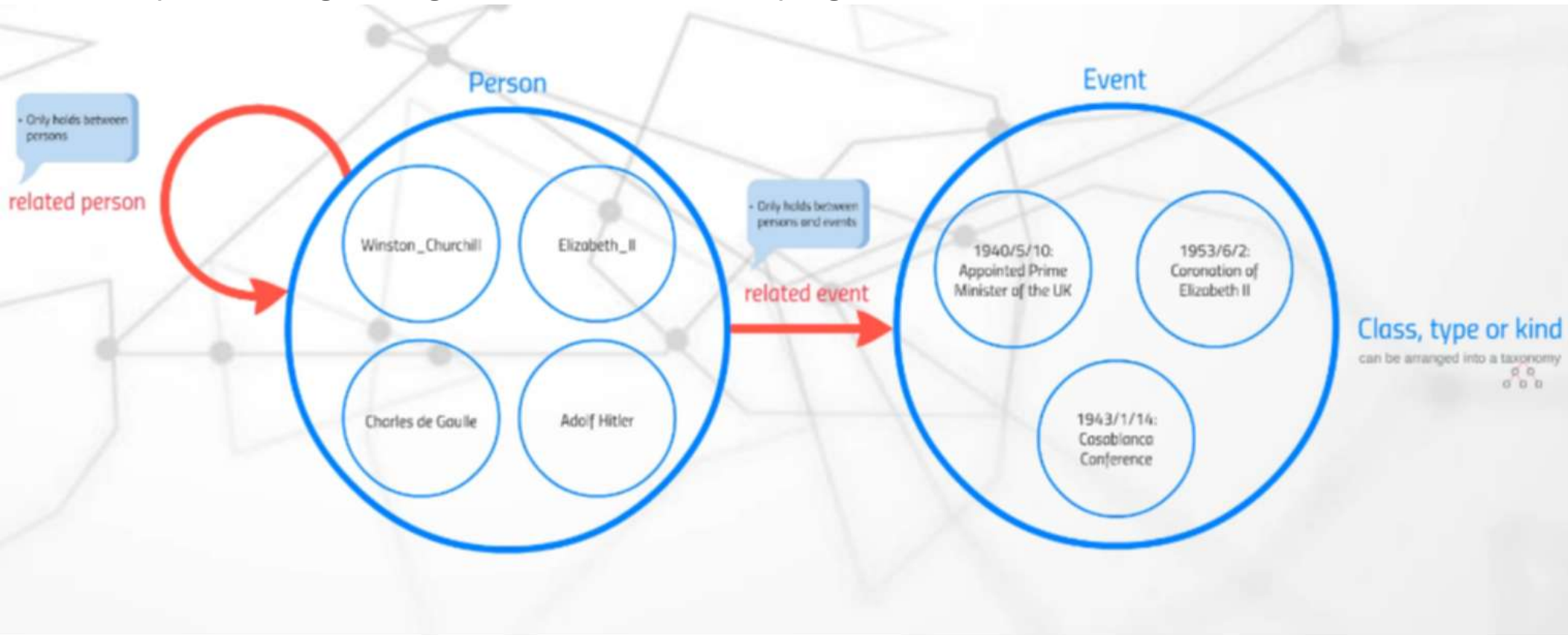
Knowledge Graph – nodes and relations





Ontology

A blueprint for organizing facts, relations. Grouping, relations and rules



Key components of a knowledge graph (a.k.a. KG)

- Nodes
 - Contain labels and properties
 - e.g., PERSON, PLACE
- Relationships (Edges)
 - Directional, e.g., FRIENDS_WITH
- Labels
 - Used to group nodes of a certain type, for e.g., all person nodes can have label Person
- Properties
 - Both nodes and relations can have properties, which are key-value pairs. For e.g. a Person node may have name="Vishal", age="30"
- Relationship Type
 - Type of relationship, for e.g. WORKS_FOR

Working with graph databases – which language?

- Cypher
 - Relationships (Edges)
 - Directional, e.g., FRIENDS_WITH
- Gremlin
 - Used to group nodes of a certain type, for e.g., all person nodes can have label Person
- Properties
 - Both nodes and relations can have properties, which are key-value pairs. For e.g. a Person node may have name="Vishal", age="30"
- Relationship Type
 - Type of relationship, for e.g. WORKS_FOR

Cypher – pattern matching language from Neo4j

Milestone	What happened	Why it matters
2012 Neo4j 2.0	Cypher introduced as a proprietary, SQL-inspired pattern-matching language.	Quickly became the best-known “property-graph” query language.
2015 openCypher	Neo4j opened the language spec and grammar under the LDBC consortium.	Vendors such as RedisGraph, Memgraph, SAP HANA Graph and Apache AGE adopted it. AWS Documentation
2024 ISO/IEC 39075 GQL	The first <i>formal</i> international Graph Query Language standard was published. It borrows syntax and semantics from openCypher, PGQL and G-CORE but is not identical to any of them. ISO Graph Database & Analytics	

Adopted by Amazon Neptune (openCypher), TigerGraph, besides of course, neo4j

Gremlin – pattern matching language from Neo4j

- Conceived in 2009 by **Marko A. Rodriguez**
- Released through the open-source **Apache TinkerPop** project, which he co-founded.
- TinkerPop positions Gremlin as a “**graph traversal machine and language**,” analogous to how the JVM hosts Java programs.

Adopted by Amazon Neptune, JanusGraph, Azure Cosmos DB

Popular graph databases

Name	Strengths	Weaknesses	Licence / Deployment model	Query-language support
Neo4j (Neo4j Inc)	• Mature ecosystem, rich tooling (APOC, Bloom, drivers) • Powerful graph algorithms & GDS library • Strong community & training resources	• Some features in enterprise-only edition (clustering, RBAC) • High memory footprint for very large graphs • GPL v3 for Community can deter closed-source redistribution	Community Edition — GPL v3; Enterprise — commercial subscription (Graph Database & Analytics)	Native Cypher (plus preview ISO GQL in v5.x)
Amazon Neptune	• Fully-managed, auto-patched & backed-up • Choice of property-graph and RDF models • Deep integration with the AWS stack	• Vendor lock-in to AWS • Separate endpoints for PG/RDF • Pricing driven by instance size + I/O	Closed-source, AWS-managed PaaS (AWS Documentation)	openCypher, Gremlin, SPARQL (AWS Documentation)
ArangoDB (ArangoDB Inc)	• Multi-model (documents + KV + graphs) in one engine • Flexible JOIN-like traversals via AQL • Good built-in clustering & sharding	• Smaller graph-specific ecosystem • Licence changed to BSL 1.1 for future versions (cloud-hosting restriction)	Apache 2.0 ≤ v3.11; BSL 1.1 (falls back to Apache 2.0 after 4 yrs) from v3.12+ (ArangoDB, ArangoDB)	Native AQL

Popular graph databases (contd.)

Name	Strengths	Weaknesses	Licence / Deployment model	Query-language support
TigerGraph (TigerGraph Inc)	• High-performance distributed engine tuned for real-time analytics • Parallel, Turing-complete GSQL language • Built-in graph data-science and vector search	• Proprietary closed source (free dev edition only) • Learning curve vs SQL/Cypher • Smaller community than Neo4j	Proprietary, node-locked licence; free single-node Dev edition (TigerGraph Documentation)	GSQL (plus optional openCypher subset) (TigerGraph Documentation)
JanusGraph (Linux Foundation)	• 100 % open-source, vendor-neutral • Back-end pluggable (Cassandra, HBase, Bigtable, etc.) • Massively scalable with external storage & index layers	• Requires external DB + search engine (ES/Solr/Lucene) • Operational complexity vs all-in-one stores • No native query DSL beyond Gremlin	Apache 2.0 (JanusGraph)	Gremlin (TinkerPop); SPARQL via plugin (JanusGraph)
Azure Cosmos DB (Gremlin API)	• Globally distributed with multi-region writes • Millisecond P99 latency SLA • Multi-model (graph, docs, KV, column) under one service	• RU/s cost model can be hard to predict • Gremlin feature set lags behind on-prem engines • Closed-source cloud service	Proprietary, fully-managed PaaS by Microsoft Azure (Microsoft Azure)	Gremlin API (plus SQL, Mongo, Table, Cassandra APIs) (Microsoft Azure)

Example – nodes, relations, properties creation using Cypher

```
CREATE (p:Person {name: "Alice", age: 30, city: "New York", email: "alice@example.com"})
```

```
CREATE (c:Company {name: "Acme Corp", location: "San Francisco", industry: "Technology"})
```

```
CREATE (p)-[r:WORKS_FOR {since: 2015, role: "Software Engineer", department: "R&D"}]->(c)
```

```
CREATE (p)-[r:FAN_OF]->(c)
```

```
CREATE (p:Person {name: "Bob", age: 30, city: "New York", email: "alice@example.com"})
```

```
CREATE (p)-[r:WORKS_FOR {since: 2015, role: "Software Engineer", department: "R&D"}]->(c)
```

- How many nodes?
- How many Node Labels?
- How many relationship types?
- How many relations?
- Nodes: 3, Node labels: 2 (Person, Company)

Cypher Queries

// Show all nodes and relationships

```
MATCH (n)-[r]->(m)
```

```
RETURN n, r, m
```

// Clear graph

```
MATCH (n) DETACH DELETE n
```

Language families used by popular graph databases

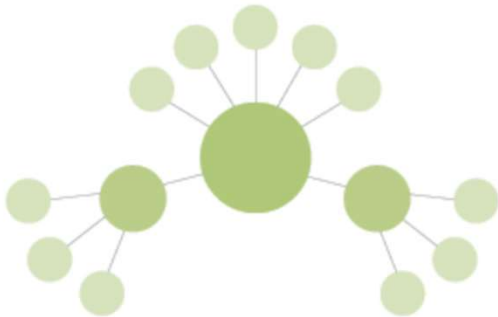
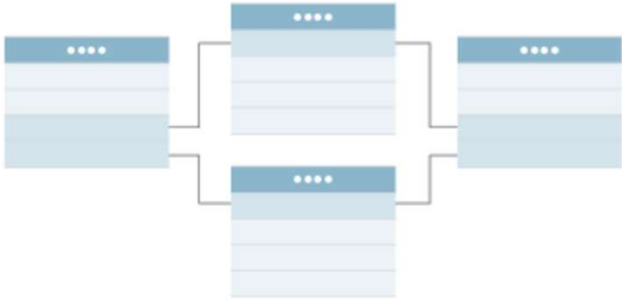
Language family	Typical database engines ¹	Notes
openCypher / Cypher	Neo4j, RedisGraph, Memgraph, SAP HANA Graph, Amazon Neptune (preview), NebulaGraph, Apache AGE	De-facto "property-graph" lingua franca, but implementation details diverge.
Gremlin (Apache TinkerPop)	JanusGraph, Azure Cosmos DB (Gremlin API), Amazon Neptune, OrientDB, DataStax Enterprise Graph	Imperative traversal style; vendor-neutral framework.
SPARQL (W3C standard)	GraphDB, Blazegraph, Virtuoso, Stardog, Neptune (RDF mode)	Targets RDF triple stores rather than property graphs.
GSQL	TigerGraph	Declarative, graph-native analytics extensions.
AQL	ArangoDB	Multi-model (documents + graphs) query language.
PGQL	Oracle PGX, Oracle Database 23c	SQL-like, one of the inputs to ISO GQL.
SQL/PGQ (part 16 of SQL:2023)	Rolling into mainstream relational DBs (e.g., PostgreSQL prototypes, Oracle 23c)	Lets you run property-graph pattern matching <i>inside</i> SQL. Peter Eisentraut ISO

Demo with neo4j

If firewall issues, use ssc to bypass certificate check

NEO4J_URI=neo4j+ssc://5df217b6.databases.neo4j.io

Graph Databases Vs RDBMS

	Graph database	Relational database
FORMAT	Nodes and edges	Tables with rows and columns
RELATIONSHIPS	Considered data, represented by edges between nodes	Related across tables, established using foreign keys between tables
COMPLEX QUERIES	Run quickly and do not require joins	Require complex joins between tables
TOP USE CASES	Relationship-heavy use cases, including fraud detection and recommendation engines	Transaction-focused use cases, including online transactions and accounting
EXAMPLE		

Neo4j Property Graph Model

Database information

Nodes (2)

 **Company**  **Person**

Relationships (1)

 **WORKS_FOR**

Property keys

age **city** **data** **department** **email** **id**

industry **location** **name** **nodes**

relationships **role** **since** **style**

visualisation

neo4j\$

```
neo4j$ MATCH (n) WHERE n.city IS NOT NULL RETI
```

Table RAW

	entity	city
1	"node"	"New York"

Started streaming 1 record after 48 ms and comp

```
1 MATCH (n)
2 WHERE n.city IS NOT NULL
3 RETURN DISTINCT "node" as entity, n.city AS city
4 LIMIT 25
5 UNION ALL
6 MATCH ()-[r]-()
```

Nested Queries

Scenario

After John Doe's X-Ray procedure in a specific visit, show me any diagnoses recorded within 30 days of that procedure.

```
// 1) Find the initial X-Ray visit for John Doe.
MATCH (p:Patient { name: "John Doe" })-[:UNDERWENT_VISIT]->(initialVisit:Visit)
      -[:HAS_PROCEDURE]->(xray:Procedure { type: "X-Ray" })
WHERE initialVisit.dateOfVisit = date("2025-01-15") // or any relevant date filter

// 2) Use WITH to pass forward the patient and the initial visit.
WITH p, initialVisit

// 3) From the same patient, find subsequent visits that have any diagnosis.
//     Only keep those visits that occurred within 30 days after the X-Ray visit.
MATCH (p)-[:UNDERWENT_VISIT]->(followupVisit:Visit)-[:HAS_DIAGNOSIS]->(diag:Diagnosis)
WHERE followupVisit.dateOfVisit > initialVisit.dateOfVisit
      AND duration.between(date(initialVisit.dateOfVisit), date(followupVisit.dateOfVisit)).days < 30

// 4) Return the details of those follow-up diagnoses
RETURN diag.name AS diagnosisName, followupVisit.dateOfVisit AS diagnosisDate
ORDER BY followupVisit.dateOfVisit
```


Nested Queries

Scenarios

After John Doe's X-Ray procedure in a specific visit, show me any diagnoses recorded within 30 days of that procedure.

Find any additional visits for the same patient after a medication was prescribed.

Show me all diagnoses that eventually lead to a referral to a specific doctor.

Within a certain time window, find subsequent procedures or medications.

Shortest Path Queries

Scenarios

Show me the shortest path between Dr. Alice (a Doctor) and Nurse Bob (a Nurse), and how many edges (hops) are between them?

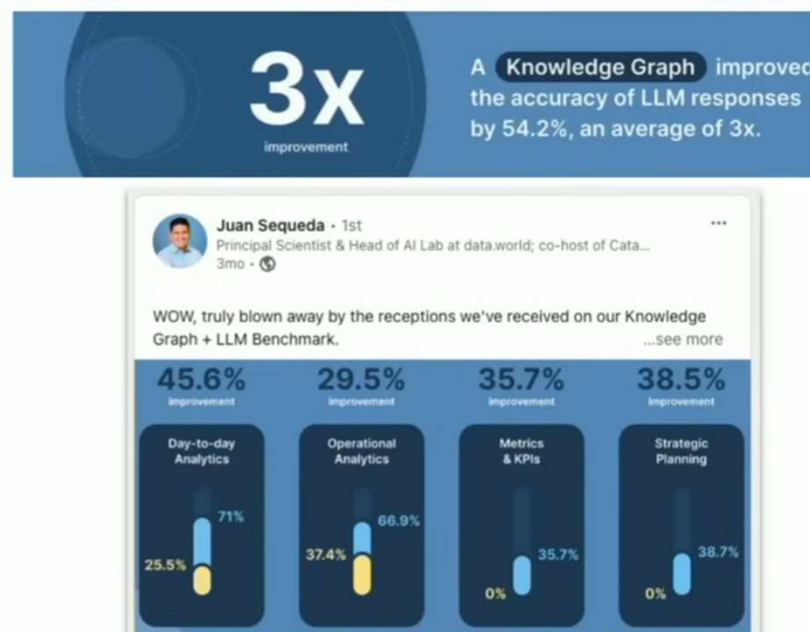
How are these two doctors connected via their referrals, shared departments, or patient handoffs?

Which chain of people (nurses/doctors) did a patient pass through before reaching the specialist?

Knowledge graph - benefits

① Higher Accuracy

The clearest driver of GraphRAG adoption amongst users is *higher accuracy*.



<https://www.youtube.com/watch?v=knDDGYHnnSI>

Knowledge graph - benefits

② Easier Development*

The second reason we hear people choose GraphRAG over vector-only RAG is *easier development*... once they've pushed through the initial learning curve.

* Weirdly, ease of development [or lack thereof] is also one of the stumbling blocks!

Feedback from an AI Engineer

I kinda replicated the same action-based cache already in place for Pinecone but thanks to the graph nature of Neo4J most of the operations better results:

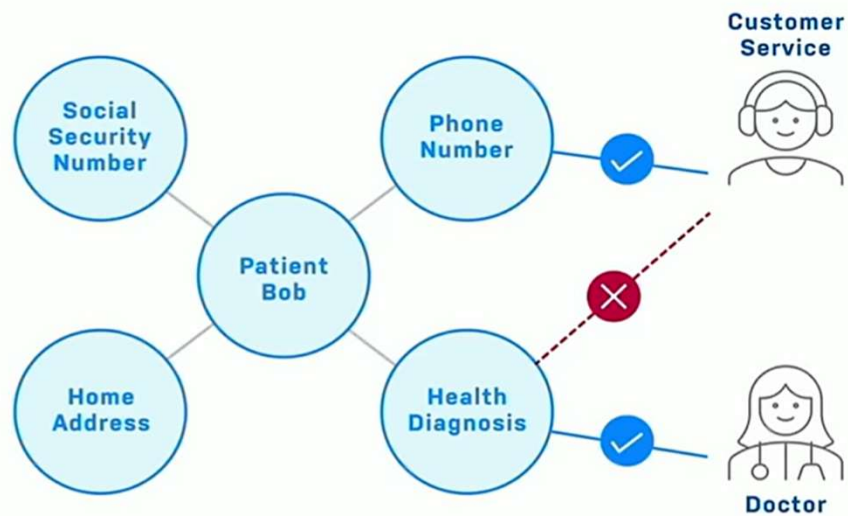
- thanks to the `Neo4j graph data science` plugin we can store embeddings and calculate cosine similarity at the database level
- getting related actions is as simple as following the relationships between nodes

Caveat: creating KG takes work, but once created ease of development is a big plus

Knowledge graph - benefits

③ Explainability & Governance

Finally, in particular central IT & business decision makers cite ease of explainability and governance.



Leveraging knowledge graph for chronological data

Usecase: medical records summarization in a chronological order

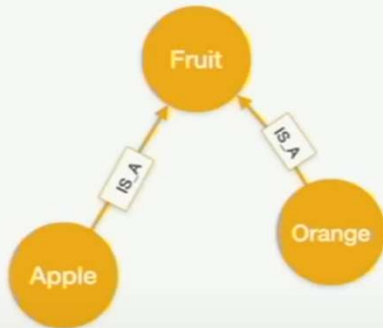
Knowledge graphs can be very useful:

- Date or timestamp can be a **property** on the MEDICAL_RECORD node
You can then use the ORDER_BY in cypher query to retrieve the records chronologically
- Temporal relationships
You can model a :NEXT_EVENT relation time to stitch the events to create a chain of events
- Temporal Queries will be fast

```
MATCH (r:MedicalRecord)
RETURN r
ORDER BY r.date ASC
```

KG Vs regular RAG

Representation



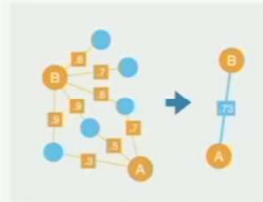
Representation

Apple	[0.2435, 3.7652, 0.00234, 456.66, ...]
Orange	[115.124, 29.7652, 4.2131, 2.431, ...]
Fruit	[0.0035, 17.661, 0.0113, 11.4566, ...]

Similarity calculation

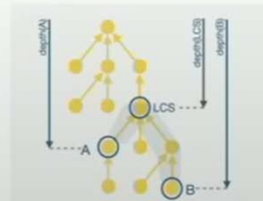
Structural

- ▶ Node similarity
- ▶ Overlap
- ▶ Jaccard



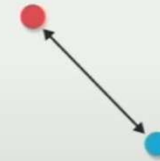
Taxonomy based

- ▶ Path
- ▶ Leacock-Chodorow
- ▶ Wu-Palmer

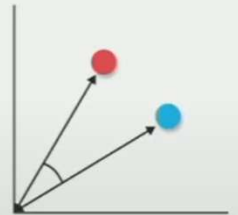


Similarity calculation

Euclidean

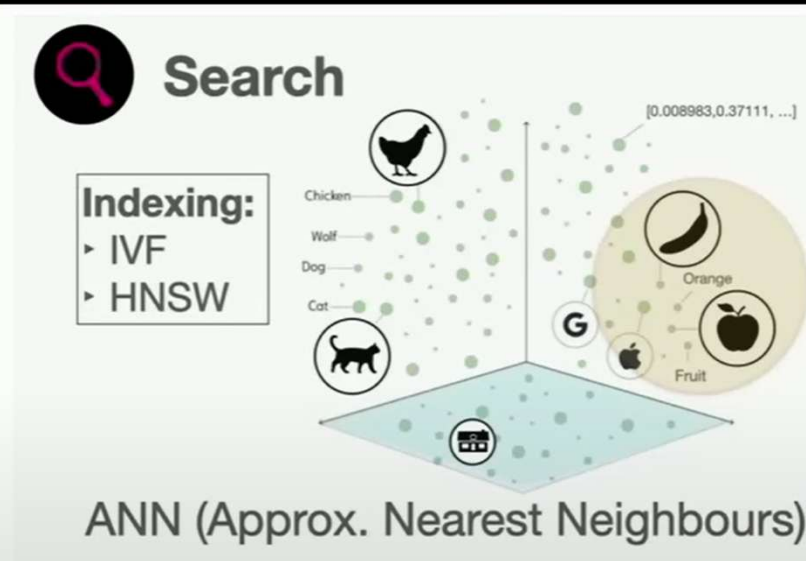
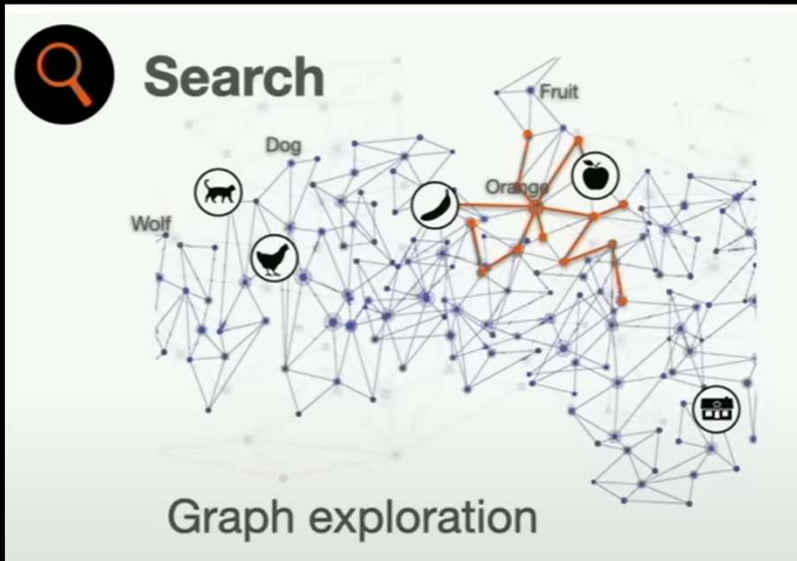


Cosine



Vector distance based

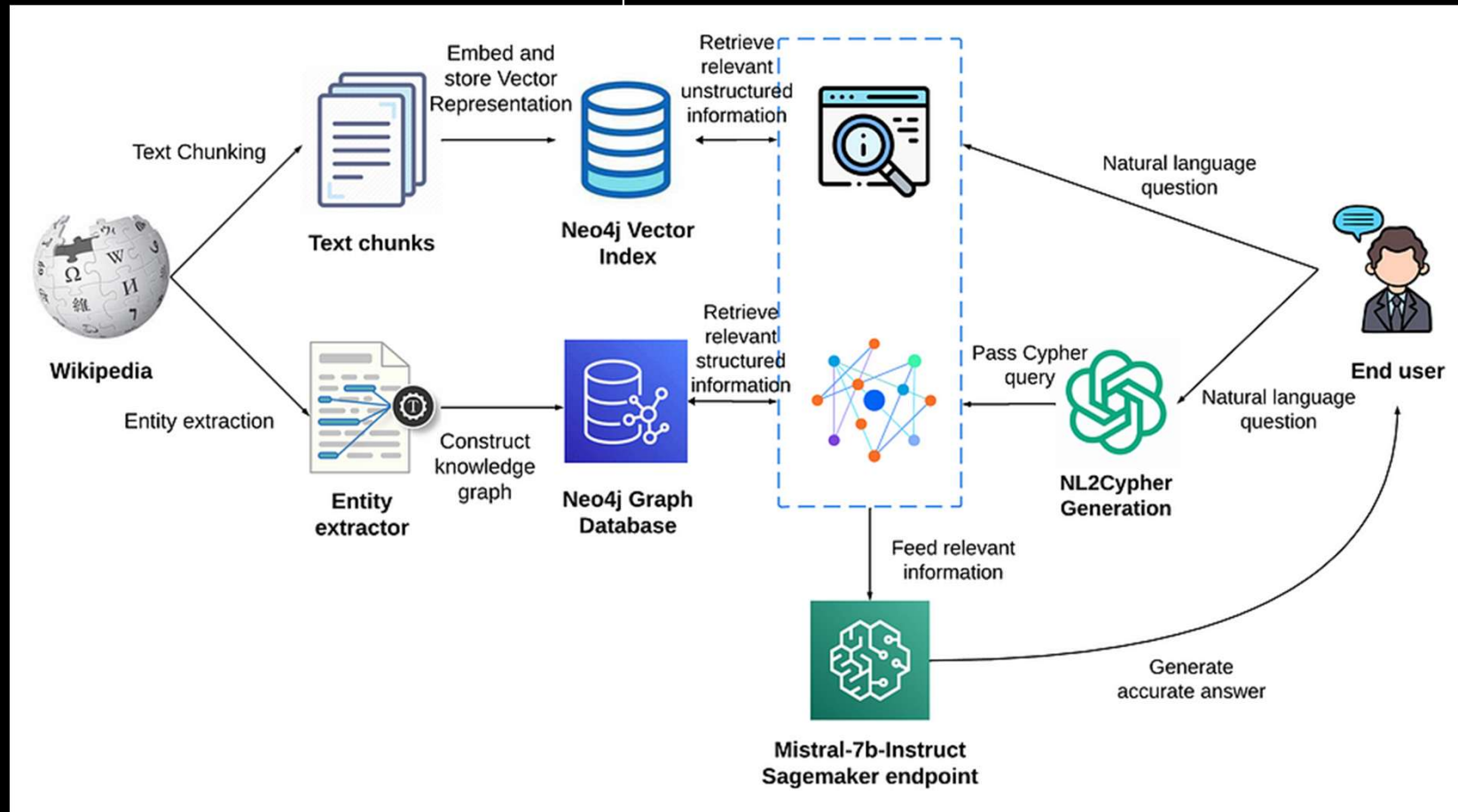
KG Vs regular RAG



Managing unstructured text and KG in neo4j

Indexing

Retrieval



Storage Architecture

Document Store (optional third component):

- Store original documents and metadata
- Maintain source references and provenance
- Examples: MongoDB, Elasticsearch, PostgreSQL, S3

Vector Database (for descriptive text):

- Store text chunks, paragraphs, sections
- Capture semantic meaning through embeddings

Graph Database (for structured knowledge):

- Store entities, relationships, and their metadata
- Maintain explicit connections between concepts
- Enable traversal and reasoning

Indexing steps with Graph Database

- Text extraction from documents (PDF for example)
- Entities extraction
- Relationships extraction
- Graph construction and storage

Graph creation tools and libraries

- Neo4j Graph Builder
- LLM
- Python libraries

Coordination strategies – Indexing

Linked storage pattern

```
# Vector chunk with graph references
vector_chunk = {
    "id": "chunk_123",
    "text": "The research team led by Dr. Smith discovered...",
    "embedding": [0.1, 0.2, ...],
    "entities": ["Dr. Smith", "research_project_X"], # Links to graph
    "chunk_position": {"page": 5, "paragraph": 2}
}

# Graph entity with vector references
graph_entity = {
    "id": "Dr_Smith",
    "type": "Person",
    "properties": {...},
    "text_chunks": ["chunk_123", "chunk_456"] # Links to vector DB
}
```

Coordination strategies – Retrieval

Phase 1: Multi modal retrieval

```
def hybrid_retrieve(query):  
    # 1. Vector similarity search  
    similar_chunks = vector_db.similarity_search(query, k=10)  
  
    # 2. Graph traversal from query entities  
    query_entities = extract_entities(query)  
    related_subgraph = graph_db.traverse_from_entities(query_entities)  
  
    # 3. Cross-reference and rank  
    combined_results = merge_and_rank(similar_chunks, related_subgraph)  
    return combined_results
```

Coordination strategies – Retrieval

Phase 2: Context assembly

```
def hybrid_retrieve(query):  
    # 1. Vector similarity search  
    similar_chunks = vector_db.similarity_search(query, k=10)  
  
    # 2. Graph traversal from query entities  
    query_entities = extract_entities(query)  
    related_subgraph = graph_db.traverse_from_entities(query_entities)  
  
    # 3. Cross-reference and rank  
    combined_results = merge_and_rank(similar_chunks, related_subgraph)  
    return combined_results
```

Integrated graph retrieval system

Initial Filtering (Keyword search)

A quick keyword search to narrow down a large corpus to a manageable set of candidates.
Such as metadata filtering

Semantic search and ranking

Apply embeddings and cosine similarity to rank the candidates based on their semantic relevance.

Relationship Analysis

Finally, incorporating graph relationships to refine or contextualize the search results, such as ranking based on node connectivity or importance.

Retrieval order and implementation options

Pattern 1: Graph-Anchored Chunks

- Store text chunks that are "anchored" to graph entities:
- Each chunk tagged with entities it discusses
- Graph entities link back to relevant chunks
- Retrieval starts with entities, expands to descriptive content

Retrieval order and implementation options

Query routing

```
def route_query(query):  
    query_type = classify_query(query)  
  
    if query_type == "factual":  
        # Start with graph, expand to text  
        return graph_first_retrieval(query)  
    elif query_type == "descriptive":  
        # Start with vector, add graph context  
        return vector_first_retrieval(query)  
    else:  
        # Balanced hybrid approach  
        return parallel_retrieval(query)
```

Retrieval order and implementation options

Fuse results

```
def fuse_results(vector_results, graph_results):  
    # Score and rank results from both sources  
    # Remove duplicates and contradictions  
    # Organize by relevance and source credibility  
    return unified_context
```